CSE3443 SOFTWARE MAINTENANCE AND EVOLUTION

CSE3443 Course Contents

- **Topic 1: Software Maintenance**
- Topic 2: Key Issues in Software Maintenance
- Topic 3: Evolution and Maintenance Model
- Topic 4: Techniques for Maintenance
- Topic 5: Software Maintenance Tools

Topic 2: Key Issues in Software Maintenance

1. Technical Issues
   - ✓ Limited Understanding
   - ✓ Testing
   - ✓ Impact Analysis
   - ✓ Maintainability
2. Management Issues
3. Maintenance Costs Estimation
4. Software Maintenance Measurement

OUTLINE OF TODAY'S LECTURE

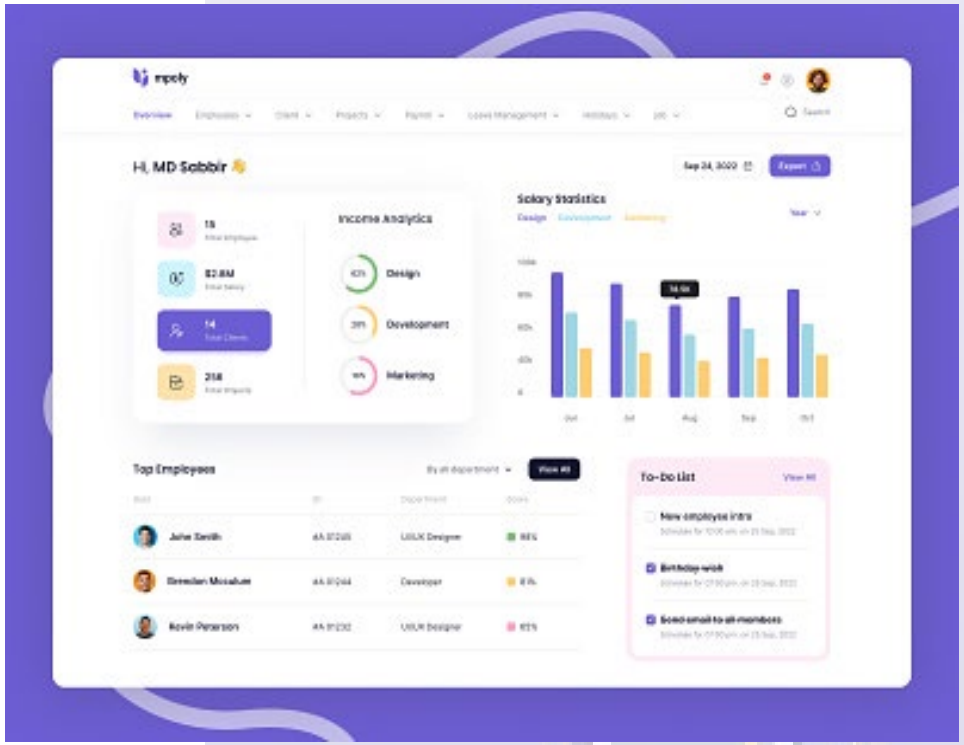
1. **Topic 2: Key Issues in Software Maintenance**
    1. **Technical Issues**
    2. **Management Issues**

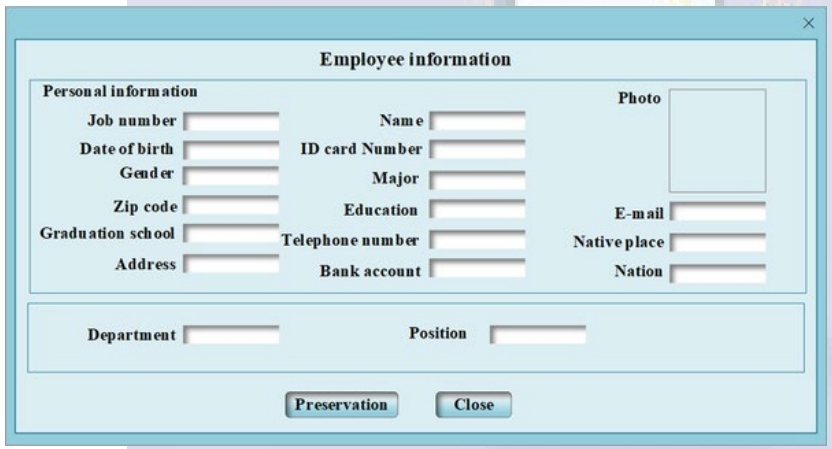
2. **Group Project Discussion**

Key Issues in Maintenance (1)

- Let’s assume that you’re given a complex system that deals with Human Resources (e.g. employee salaries, employee leaves etc)
- You need to maintain/modify the system to meet its current needs.
- You’re new to the company and were not involved in the development of the system.
- You’ve limited understanding of the original code.

Question #1: What are some of the challenges that can arise in maintaining the HR system?





Question #1:
What are some of the challenges that can arise in maintaining the system, when there is a limited understanding of the original code

1. It can be difficult to identify the root cause of bugs or errors
2. It can be challenging to modify or update the software
3. Challenging to effectively communicate with stakeholders

Question #2:
How can these challenges impact the effectiveness and reliability of the software over time?

Question #2:
How can these challenges impact the effectiveness and reliability of the software over time?

1. It can be difficult to identify the root cause of bugs or errors
2. It can be challenging to modify or update the software
3. Challenging to effectively communicate with stakeholders

1. leading to extended downtime and reduced productivity.
2. introducing unintended consequences or new bugs.
3. difficult to explain why certain changes are necessary, or to provide accurate estimates of the time and resources needed to complete maintenance tasks

Real world example

In 2018, Facebook experienced a major software bug that exposed the private information of millions of users.

The bug was caused by a software maintenance issue, as Facebook developers had modified the code of a feature that allowed users to see their own profile as if they were someone else.

This modification introduced a bug that allowed attackers to steal access tokens, which could be used to access user accounts without needing a password.

The bug was fixed quickly, but it raised concerns about the security and reliability of Facebook's software.

Limited Understanding

Question #3:
What contributes to the limited understanding of code?

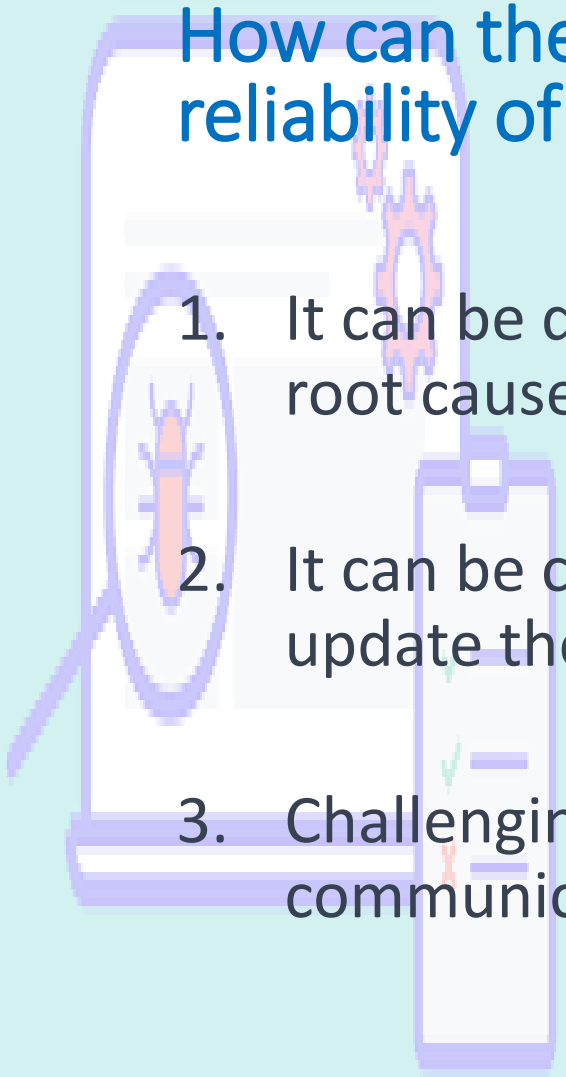
- Incomplete documentation that makes it hard to understand the software.
- Lack of knowledge and experience in software maintenance.

- How to deal with this issue?
  - Identifies potential sources of limited understanding, such as lack of documentation or knowledge transfer
  - Offers tips for improving understanding and knowledge transfer in software maintenance

Key Issues in Maintenance (2)

Testing issues

Question #2:
How can these challenges impact the effectiveness and reliability of the software over time?



1. It can be difficult to identify the root cause of bugs or errors
2. It can be challenging to modify or update the software
3. Challenging to effectively communicate with stakeholders

1. leading to extended downtime and reduced productivity.
2. introducing unintended consequences or new bugs.
3. difficult to explain why certain changes are necessary, or to provide accurate estimates of the time and resources needed to complete maintenance tasks

Key Issues in Maintenance (2)

- Key challenges in maintenance related to testing.

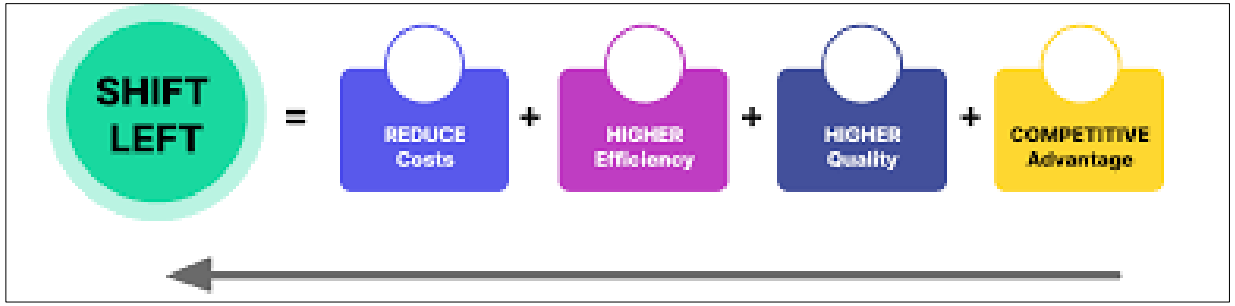
1. Inadequate testing
   - where testing is not done thoroughly or is focused on only a subset of the software's functionality.
   - This can result in missed bugs or errors, which can lead to reduced productivity and extended downtime.

2. Ineffective test plans
   - where test plans may not be designed effectively and miss critical functionality or may not be specific enough to identify potential bugs or errors.

3. Difficulties in reproducing bugs or errors
   - especially if they only occur under specific circumstances or are the result of complex interactions between different components of the software.

- It is essential to address these challenges through ongoing attention to testing and the development of effective test plans that cover all critical aspects of the software.

Testing



- It is important to implement efficient and effective testing procedures to detect and fix bugs, reduce software maintenance time, and enhance software quality

1. Introduces testing as a critical aspect of software maintenance
2. Describes the different types of testing, such as unit testing and integration testing
3. Discusses the importance of testing for identifying technical issues and improving maintainability
4. Outlines best practices for testing in software maintenance
5. Lack of proper testing leads to undetected defects and bugs.
6. Inefficient testing process that delays software maintenance.

Impact Analysis and Maintainability

- Important to improve the maintainability of software through proper design and implementation, and to conduct impact analysis to predict the effect of changes on software systems.
    1. Emphasizes maintainability as a crucial factor in software maintenance
    2. Defines maintainability and explains its importance in ensuring software longevity
    3. Discusses the different types of maintenance related to maintainability, such as corrective, preventative, and perfective maintenance
    4. Offers practical tips for improving maintainability, such as modular design and code documentation
    5. Inability to predict the impact of changes to software systems.
    6. Poor maintainability that makes software maintenance difficult.

Class Activity

- **Communication Breakdown during Change Implementation**
  - Scenario:
    - A large-scale software upgrade is planned for a multinational corporation's enterprise systems.
    - Despite comprehensive planning and coordination, communication breakdowns occur between the development team, operations team, and end-users.
    - Critical information regarding the timing, impact, and rollout of changes is not effectively communicated, leading to confusion, resistance, and operational disruptions.

- Let’s explore potential solutions to the problem

Class Activity

- **Communication Breakdown during Change Implementation**
  - Scenario:
    - A large-scale software upgrade is planned for a multinational corporation's enterprise systems.
    - Despite comprehensive planning and coordination, communication breakdowns occur between the development team, operations team, and end-users.
    - Critical information regarding the timing, impact, and rollout of changes is not effectively communicated, leading to confusion, resistance, and operational disruptions.

  - Potential Solution:
    - The project manager must prioritize transparent and proactive communication channels, establish clear escalation paths for issues, and conduct thorough change impact analysis to anticipate potential disruptions. Implementing change management processes, such as regular status updates, user training sessions, and post-implementation reviews, can foster better alignment and minimize the impact of changes.

Management Issues

- Important to establish proper management procedures and allocate sufficient resources to ensure efficient and effective software maintenance while minimizing maintenance costs.

1. Lack of proper planning and management of software maintenance activities.
2. Insufficient budget allocation for software maintenance.

Topic 2: Key Issues in Software Maintenance

1. Technical Issues
   - ✓ Limited Understanding
   - ✓ Testing
   - ✓ Impact Analysis
   - ✓ Maintainability
2. Management Issues
3. Maintenance Costs Estimation
4. Software Maintenance Measurement

Warm Up #1 ☺

What **metrics** do you know in relation to software development activities?

Productivity metric: measure of performance

Quality metrics:

ChatG

1. **Lines of Code (LOC)**: Measures the size of the codebase. However, this metric can be misleading as it doesn't account for code complexity or quality.
2. **Cyclomatic Complexity**: Measures the complexity of a program by counting the number of independent paths through the code. High cyclomatic complexity can indicate code that is difficult to maintain and test.
3. **Code Churn**: Measures the rate at which code is added, modified, or deleted over time. High code churn may indicate instability in the codebase.
4. **Defect Density**: Measures the number of defects (bugs) per unit of code. This helps assess the quality of the codebase.
5. **Code Coverage**: Measures the percentage of code that is covered by automated tests. Higher code coverage generally indicates a more thoroughly tested codebase.
6. **Lead Time**: Measures the time it takes for a new feature or change to go from concept to production. Shorter lead times often indicate higher efficiency.
7. **Cycle Time**: Measures the time it takes for a task to move through the development process (e.g., from development to testing to deployment). Shorter cycle times indicate faster development cycles.
8. **Velocity**: In Agile development, velocity measures the amount of work completed by a team in a sprint. It helps teams plan future sprints and track progress over time.
9. **Customer Satisfaction**: Measures how satisfied customers are with the software product. This can be measured through surveys, feedback, and other qualitative methods.
10. **Uptime/Availability**: Measures the percentage of time that the software system is available and functioning correctly. High uptime is critical for systems that need to be available 24/7.

Warm Up #2 ☺

Categorize the software development metrics according to different stages of the development life cycle, in a table.

……

ChatGPT says….

Development Life Cycle Stage	Metrics
Requirements Gathering and Analysis	- Customer Satisfaction - Requirements Volatility
Design Phase	- Cyclomatic Complexity - Design Review Defect Density
Development Phase	- Lines of Code (LOC) - Code Churn - Defect Density - Code Coverage
Testing Phase	- Test Coverage - Defect Detection Rate - Mean Time to Detect (MTTD)
Deployment and Operations	- Uptime/Availability - Mean Time to Recovery (MTTR) - Deployment Frequency
Post-Deployment/Maintenance	- Customer Satisfaction - Mean Time Between Failures (MTBF) - Mean Time to Resolution (MTTR)

#1 – Time spent supporting production

- The total time that the maintenance team spends on production-focused activities.

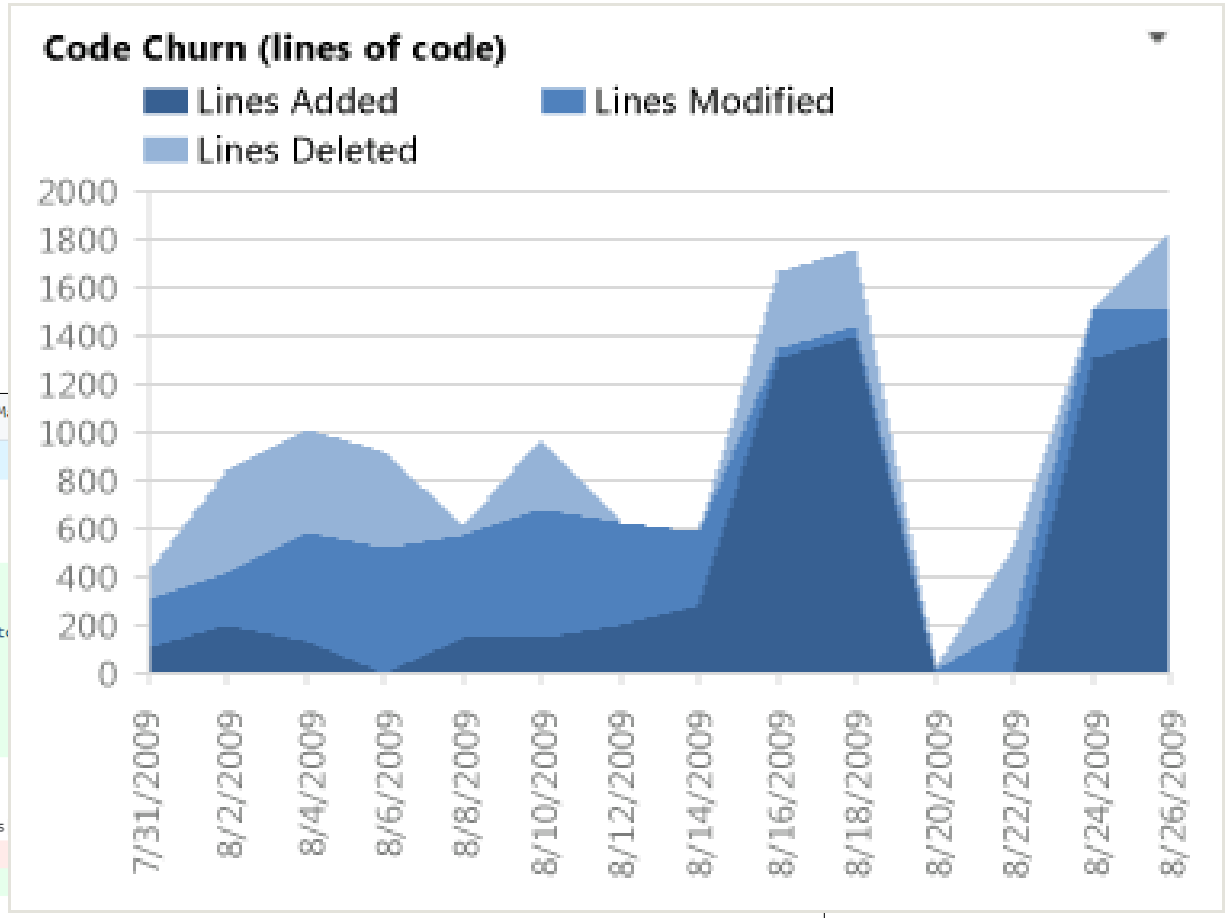
- Usually measured weekly, monthly, or quarterly.

#2 – Code Churn

- Code churn represents the number of lines of code that were modified, added or deleted in a specified period of time.

- If code churn increases, then it could be a sign that the software development project needs attention.

#2 – Code Churn



Filter changed files

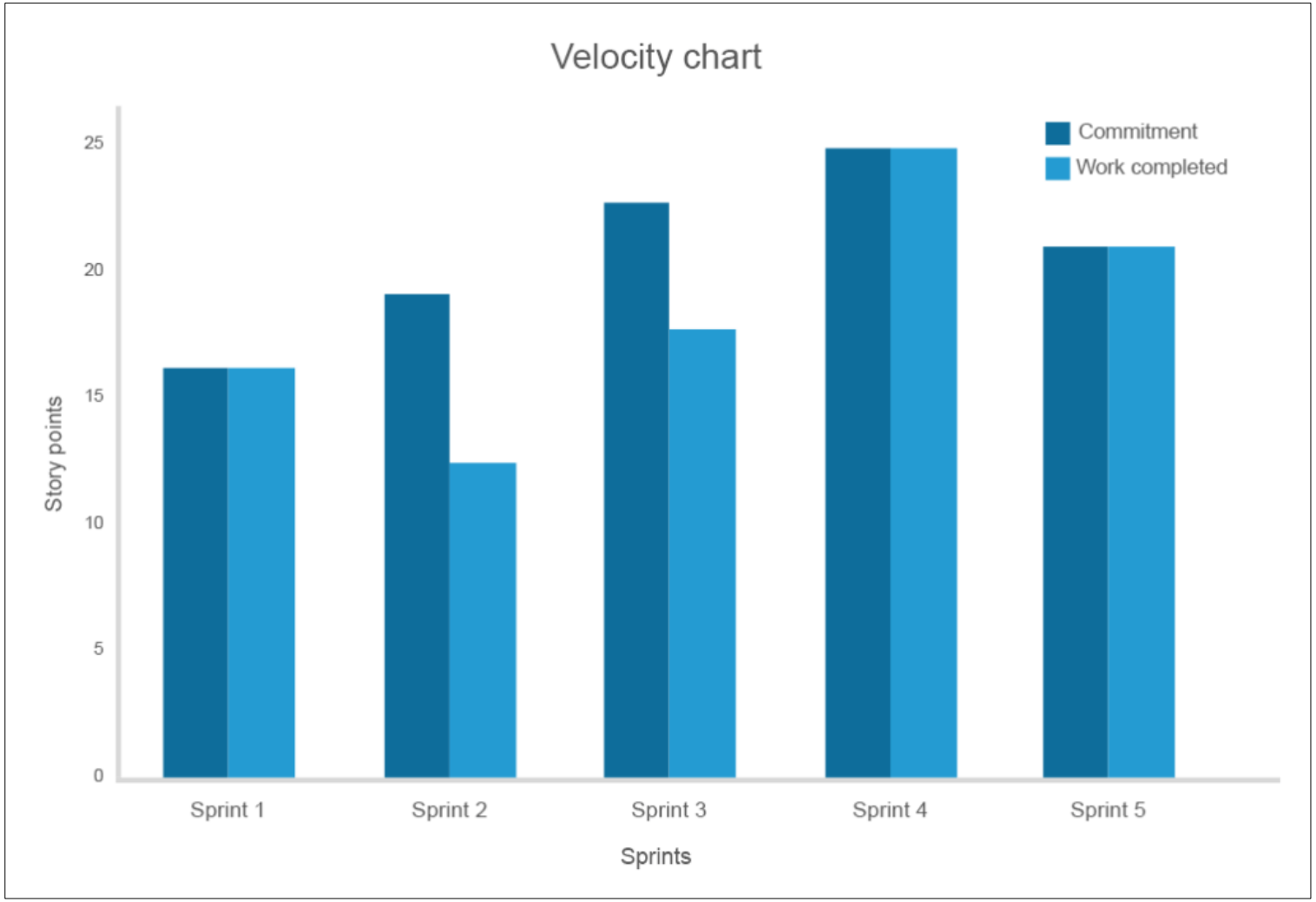
- .idea
  - misc.xml
- CoffeeMaker/CoffeeMaker_JUnit/...
  - Main.java

16 CoffeeMaker/CoffeeMaker_JUnit/src/edu/ncsu/csc326/coffeemaker/M

```
@@ -102,17 +102,25 @@ public static void addRecipe() {
102 102          */
103 103      public static void deleteRecipe() {
104 104          Recipe [] recipes = coffeeMaker.getRecipes();
105     +
106     +        if(recipes[0]==null){
107     +                System.out.println("There are no recipe t
108     +                mainMenu();
109     +
110     +        }
111     +
105 112          for(int i = 0; i < recipes.length; i++) {
106 113                  if (recipes[i] != null) {
107 114                          System.out.println((i+1) + ". " + recipes
108         -                Todelete();
115     +
109 116                  }
110 117                  else {
111         -                System.out.println("There are no recipe to be deleted.\n");
112         -                mainMenu();
113         -
118     +                        Todelete();
119     +
114 120                  }
121     +
115 122          }
123     +
116 124
117 125      }
118 126      public static void Todelete() {
```

#3 – Velocity Chart

- Velocity measures the amount of work (a number of features) completed in a sprint.
- While it isn't a prediction or comparison tool, velocity provides teams with an idea about how much work can be done in the next sprint.



Introduction to Measurement

- Software metrics are a set of measures used to quantitatively assess and evaluate software products and processes.
- They are essential in software development for measuring software performance, planning work items, measuring productivity, and many other uses.
- Software metrics are valuable for providing insights into software quality, improving the software development process, and helping in decision-making.
- They also help in identifying problem areas in software development and monitoring the effectiveness of changes made to the software development process

Why are Software Metrics Important?

- Establish baselines for comparison with future assessments.
- To evaluate the status with respect to plans
- Predict qualities by gaining understanding of relationships among process and products.
- Improvement

Types of software metrics

PROCESS METRICS	PRODUCT METRICS	PROJECT METRICS
• Time in locating defect(s). • Resolving Time • Response Time • * • * • * • * • *	• Size and Design. • Complexity involved. • Performance • Usability • Security • * • * • * • *	• Number of Teams. • Number of Developers. • Time & Cost of the Project. • * • * • * • *

Metrics for Maintenance

- Maintenance metrics are used to measure and evaluate the effectiveness of a maintenance program from a management perspective.
-
- They help organizations understand the reliability and efficiency of their equipment and provide a basis for comparing the performance of different systems and equipment.

- There are various maintenance metrics available that can be used to track progress towards achieving specific goals.

Some of the most important maintenance metrics

Mean Time Between Failure (MTBF):

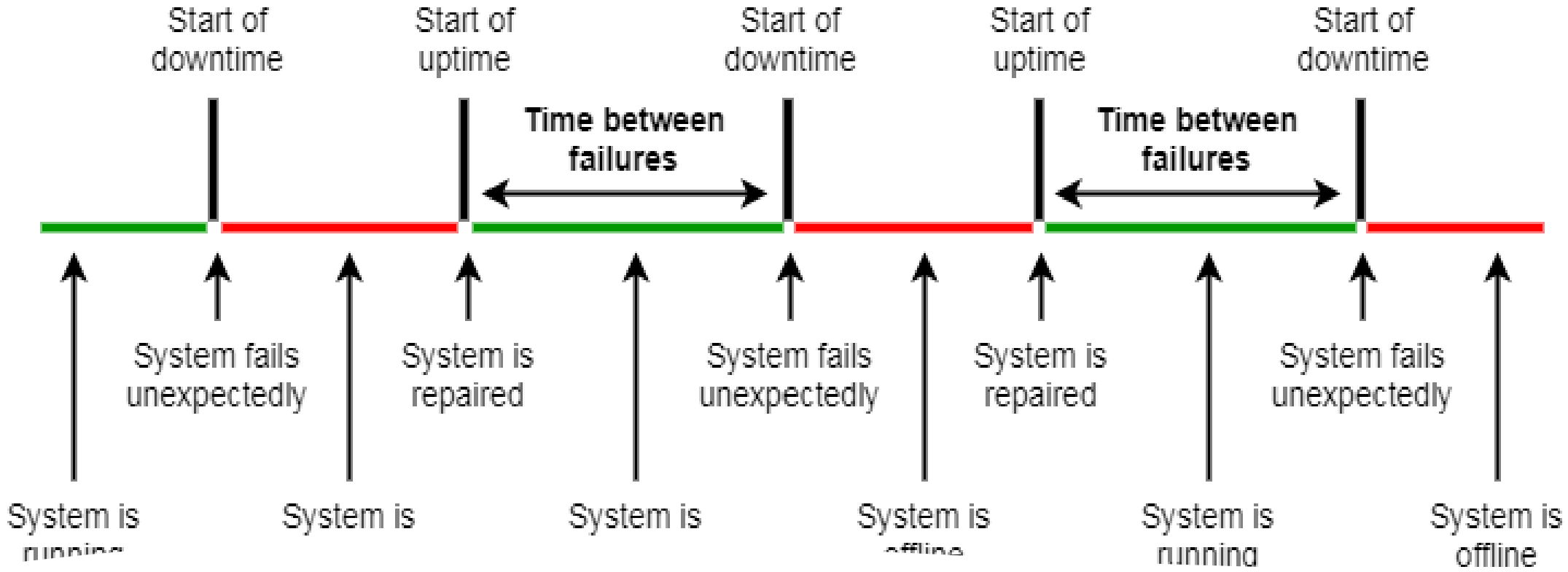
- MTBF refers to the average time between breakdowns.
- This metric is used to analyze the equipment design and safety of complex assets. It is based on the ratio of operational hours to the number of failures

Mean Time To Repair (MTTR):

- MTTR is the average time it takes to repair a failed component or system.
- It is used to analyze the effectiveness of maintenance teams and can help identify areas for improvement

Overall Equipment Effectiveness (OEE):

- OEE is a comprehensive metric that measures the overall efficiency of equipment.
- It takes into account factors such as availability, performance, and quality

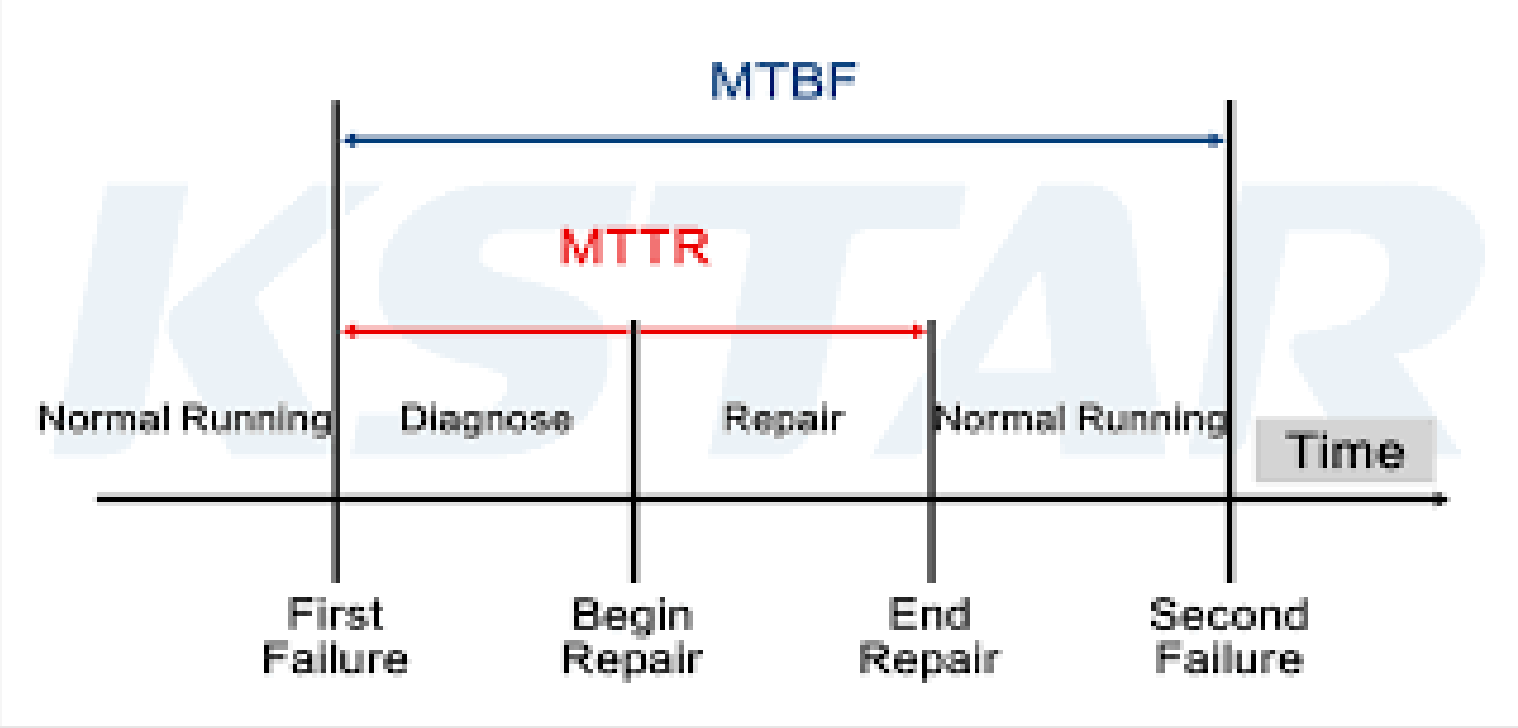


Mean Time Between Failure (MTBF):

- MTBF refers to the average time between breakdowns.
- This metric is used to analyze the equipment design and safety of complex assets. It is based on the ratio of operational hours to the number of failures

Mean Time To Repair (MTTR):

- MTTR is the average time it takes to repair a failed component or system.
- It is used to analyze the effectiveness of maintenance teams and can help identify areas for improvement



Benefits and limitation of metrics

Some benefits of maintenance metrics include:

- Improved equipment reliability and availability
- Increased productivity and efficiency
- Reduced maintenance costs
- Better decision-making and resource allocation

Limitations to maintenance metrics.

- some metrics may not provide a complete picture of overall performance or may be difficult to measure accurately.
- Additionally, metrics should be carefully chosen and customized to fit the specific needs and goals of an organization

Topic 2: Key Issues in Software Maintenance

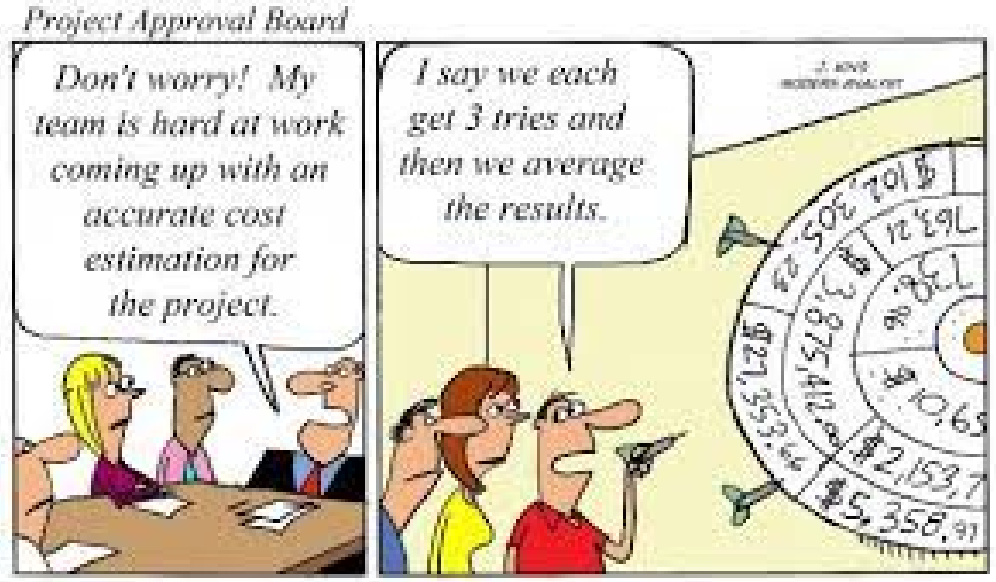
1. Technical Issues
   - ✓ Limited Understanding
   - ✓ Testing
   - ✓ Impact Analysis
   - ✓ Maintainability
2. Management Issues
3. Maintenance Costs Estimation
4. Software Maintenance Measurement

The importance of software maintenance costs estimation

- Organizations are concerned about the costs of software maintenance, as they have been increasing over time.
- Cost estimation helps in forecasting the resources and associated costs required to execute a project, ensuring that project objectives are achieved within the approved timeline and budget.

Major concerns about maintenance

- During planning phase of a software project, some main concerns are:
  - to estimate the number of maintenance programmers that will be needed
  - specifying the facilities required for maintenance



Maintenance estimation rule

- Understanding the different types of maintenance, such as adaptive, corrective, preventive, and perfective, can help in estimating maintenance costs and determining the level of effort required for each type.

- A widely used rule of thumb for the distribution of maintenance activities is:
  - 60% - Enhancement
  - 20% - Adaptation
  - 20% - Corrections

Factors affecting maintenance costs

- Complexity of the software
- Size of the software
- Quality of the software
- Technology used to develop the software
- Skills of the maintenance team

Different components that make up software maintenance costs

- Software maintenance costs are expenses
  - modifying a software product after delivery to correct faults, improve performance or other attributes, or adapt the product to a modified environment.
- What makes up the maintenance costs?

Maintenance cost components

Labor costs

- Cost of the personnel who perform the maintenance, such as software developers, engineers, and technicians.
- The cost may vary depending on the qualifications, skills, and experience of the personnel.

Hardware and software costs

- Cost of hardware and software tools used for maintenance, such as servers, software licenses, and development tools.
- The cost may also include the cost of hardware upgrades or replacement required for maintenance purposes.

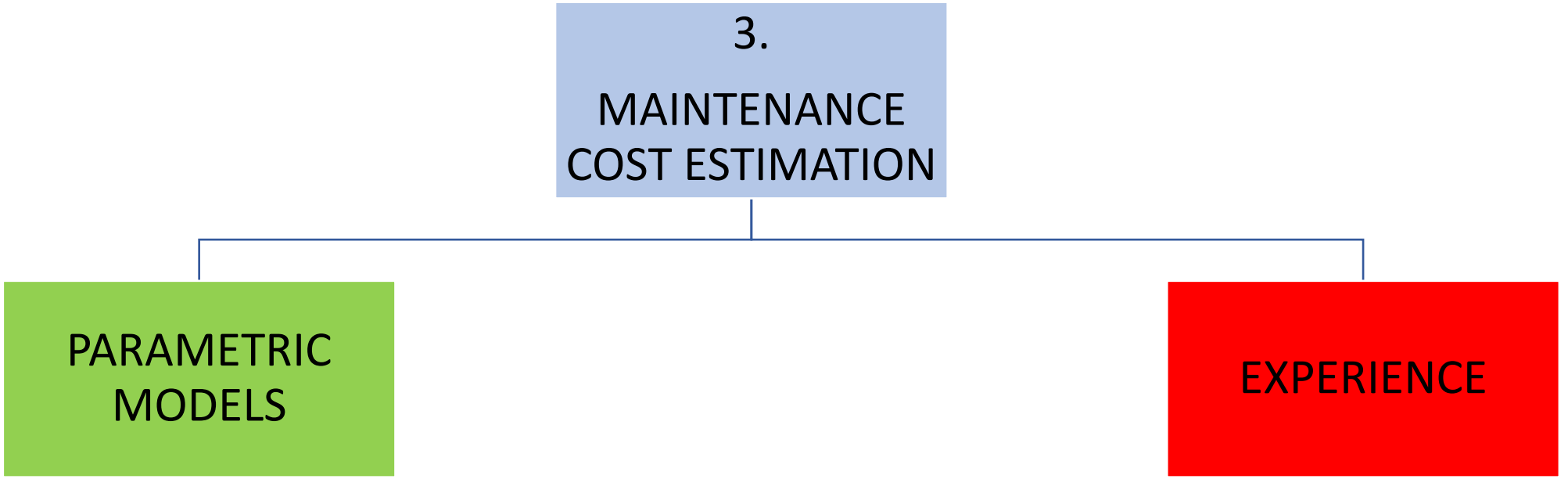
Change Request Management

- Involves identifying the need for a change in the software product.
- Involves identifying and describing the status of all the change requests made.
- All changes made to the software product are properly tracked and documented.

Impact Analysis

- Involves identifying the potential impact of the change requests on the software product.
- Determine the resources needed for the change requests, including the personnel, hardware, and software costs, and the time required for the changes.

Techniques for estimating maintenance costs

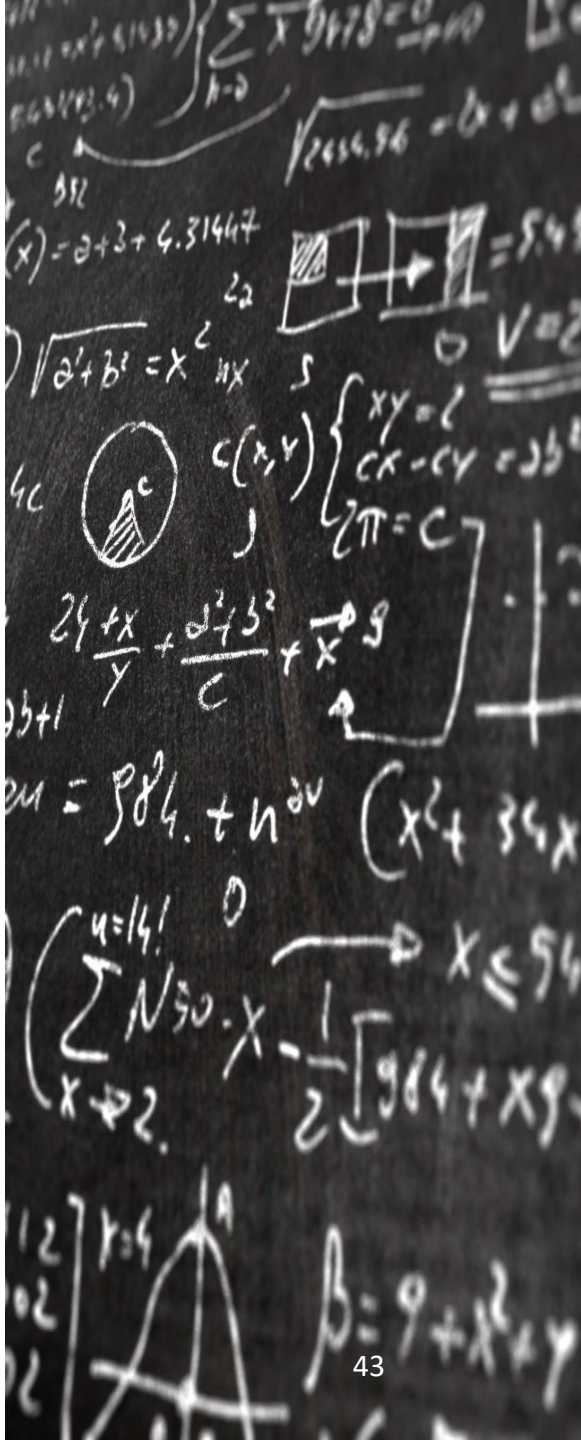


- Mathematical models
- Historical data analysis
- Algorithmic cost models

- Expert judgment – estimate maintenance effort
- Combination experience and historical data – best approach

Parametric Model: Algorithmic Cost Models

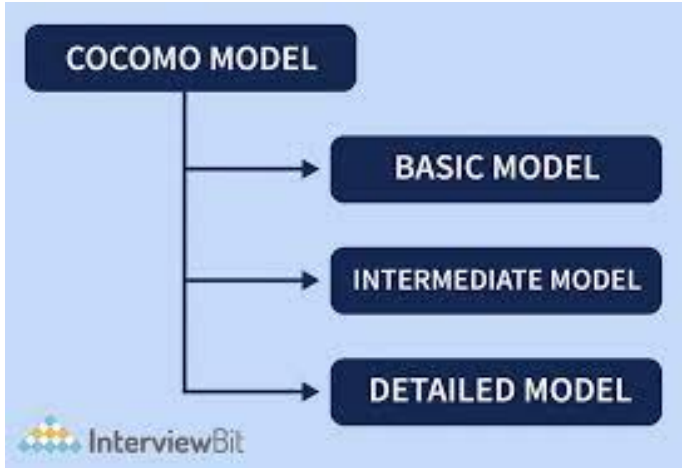
- A mathematical function is used to estimate the cost.
- The model uses inputs such as: Project, Software Process and Software Product attributes to derive the function that predicts the cost.
- Decided by project managers.
- The model is based on historical data
  - Commonly used product attribute is Lines of code (code size)
- Algorithmic cost models are scientific and objective but not necessarily the most accurate method for estimating software maintenance costs.



COCOMO (Constructive Cost Model)

- A widely used algorithmic cost model that estimates the effort required to develop or maintain software based on lines of code.

- COCOMO uses three different models, with different level of detail and accuracy:
  - basic,
  - Intermediate
  - Advanced



- Other algorithmic cost models:
  - SLIM (Software Life Cycle Management)
  - SEER (Software Estimation and Evaluation).

- These models differ in terms of the software metrics they use and the level of detail they provide.

Basic COCOMO Model: Formula

The basic COCOMO equation

- $E = a_b (KLOC \text{ or } KDSI)^{b_b}$
- $D = c_b (E)^{d_b}$
- $P = E/D$ where
  - **E** is the effort applied in person-months,
  - **D** is the development time in months,
  - **KLOC** / **KDSI** is the estimated number of delivered lines of code for the project (expressed in thousands)
  - **P** is the number of people required and
  - a_b, b_b, c_b and d_b are coefficients given in next slide.

Thank you